

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

1. Функции. Видове.

Функцията е подпрограма, която се състои от набор от инструкции, използвани за изпълнение на конкретна задача.

А. Вградени

- ❑ Това са функциите, които вече са създадени и съхранени в Python. Те са достъпни за използване от програмиста.
- ❑ Те не могат да се променят.

Б. Потребителски дефинирани

❑ Дефиниране на потребителски функции

Дефиницията на функция определя името на нова функция и последователността от изрази, които се изпълняват при извикване на функцията.

Синтаксис:

`def function name(parameter list):` — Function header

`Statement-1`
`Statement-2`
`⋮`
`⋮`
`Statement-n` } — Body of the function

- ✓ ключова дума **def** последвана от името на функцията и аргументите и, поставени в скоби, след което се поставя двоеточие.
- ✓ следва блок от програмни оператори започващ на нов ред, задължително с отстъп.
- ✓ наименоване на функция е по същият начин, както наменуване на променлива.
- ✓ първият ред от дефиницията на функцията се нарича **хедър**, останалата част е тяло на функцията.
- ✓ хедъра трябва да завършва с двоеточие, а тялото трябва да бъде с отстъп.
- ✓ функцията съдържа произволен брой изрази.
- ✓ списъкът с параметри съдържа списък със стойности, използвани във функцията.

❑ Извикване на потребителска функция

Пример:

Програма илюстрираща
използването на дефинирани
от потребителя функции

def add_numbers(x,y):

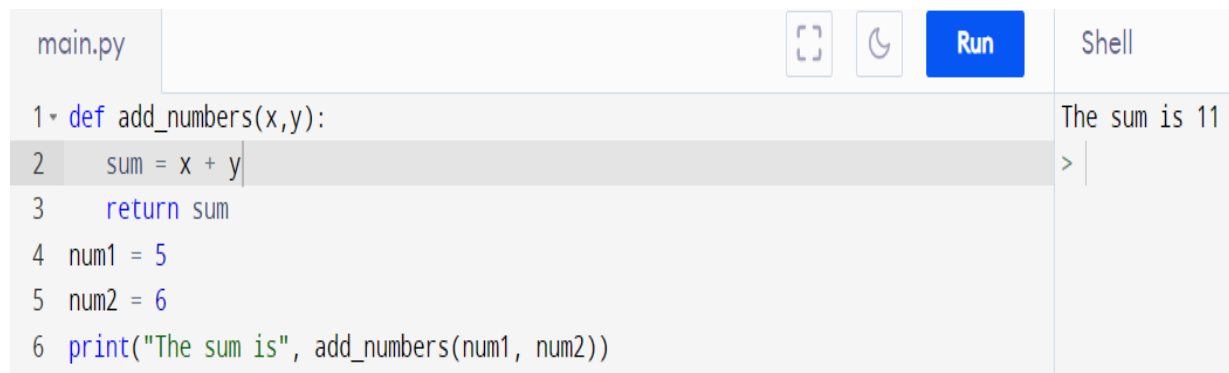
sum = x + y

return sum

num1 = 5

num2 = 6

print("The sum is", **add_numbers(num1, num2)**)



```
main.py
1 def add_numbers(x,y):
2     sum = x + y
3     return sum
4 num1 = 5
5 num2 = 6
6 print("The sum is", add_numbers(num1, num2))
```

Run Shell

The sum is 11

❑ Предимства от използване на потребителски функции

- създаване на собствени функции;
- декомпозиране на големи програми, което улеснява разбирането, поддръжка и отстраняването на грешки;
- избягва се повторно използване на кода;
- намалява се обема на програмата;
- улеснява работа при големи проекти, спомага за разделяне на натоварването, като се разпределя създаването на функции от различни програмисти.

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Таблица 1. Вградени функции

Вградени функции	Описание
>>>max(3,4) 4	# връща най-големия елемент
>>>len("hello") 5	# връща дължината на обект
>>>range(2,8,1) [2, 3, 4, 5, 6, 7]	# връща диапазон от дадени стойности (от 2 до 8 със стъпка 1)
>>>round(7.8) 8.0	# връща закръглено цяло число от даденото число
>>>chr(102) f	# връща символ (низ) от цяло число
>>>float(5) 5.0	# връща плаващо число от низ или цяло число
>>>int(5.0) 5	# връща цяло число от низ или float
>>>pow(3,5) 243	# връща число повдигнато на степен
>>>type(5.6) <type 'float'>	# връща тип данни на обекта, към който принадлежи
>>>t=tuple([4,6.0,7]) (4, 6.0, 7)	# създава набор от елементи от списъка
>>>print("good morning") Good morning	# показва дадения обект

Таблица 2. Списък с функции в Python

Списък с функции в Python Math Module	
Функция	Описание
<code>ceil(x)</code>	Връща най-малкото цяло число, по-голямо или равно на x .
<code>copysign(x, y)</code>	Връща x със знака на y
<code>fabs(x)</code>	Връща абсолютната стойност на x
<code>factorial(x)</code>	Връща факториала на x
<code>floor(x)</code>	Връща най-голямото цяло число, по-малко или равно на x
<code>fmod(x, y)</code>	Връща остатък, когато x се раздели на y
<code>frexp(x)</code>	Връща мантисата и степента на x като двойка (m, e)
<code>fsum(iterable)</code>	Връща точна сума от стойности с плаваща запетая в
<code>iterable</code>	
<code>isfinite(x)</code>	Връща <code>True</code> , ако x не е нито безкрайност, нито NaN
(Not a Number)	
<code>isinf(x)</code>	Връща <code>True</code> , ако x е положителна или отрицателна
безкрайност	
<code>isnan(x)</code>	Връща <code>True</code> , ако x е NaN
<code>ldexp(x, i)</code>	Връща $x * (2^{**i})$
<code>modf(x)</code>	Връща дробната и целата част на x
<code>trunc(x)</code>	Връща съкратената целочислена стойност на x
<code>exp(x)</code>	Връща e^{**x}
<code>expm1(x)</code>	Връща $e^{**x} - 1$

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Синтаксис на range():

range(stop)

range(start, stop, step)

Пример:

Приложение на

функцията range()

за отпечатване на число

for i in **range(10):**

 print(i, end=" ")

print()

използване на range за итериране

l = [10, 20, 30, 40]

for i in **range(len(l)):**

 print(l[i], end=" ")

print()

намиране на сбор на естествени числа

sum = 0

for i in **range(1, 11):**

 sum = sum + i

print("Sum of first 10 natural number :", sum)

```
1 # Python Program to
2 # show range() basics
3 # printing a number
4 for i in range(10):
5     print(i, end=" ")
6 print()
7
8 # using range for iteration
9 l = [10, 20, 30, 40]
10 for i in range(len(l)):
11     print(l[i], end=" ")
12 print()
13
14 # performing sum of natural
15 # number
16 sum = 0
17 for i in range(1, 11):
18     sum = sum + i
19 print("Sum of first 10 natural number :", sum)
20
```

Резултат:

```
0 1 2 3 4 5 6 7 8 9
10 20 30 40
Sum of first 10 natural number : 55
>
```

В. Видове функции

Въз основа на аргументи и тип за връщане, функциите се класифицират в 4 типа.

- ☐ *Функция без аргументи и без тип за връщане;*
- ☐ *Функция с аргументи и без тип за връщане;*
- ☐ *Функция без аргументи и с тип за връщане;*
- ☐ *Функция с аргументи и с тип за връщане.*

Функция без аргументи и без тип за връщане

В този тип не се предава аргумент през извикването на функцията и не се връща изход към основната функция. Подфункцията ще прочете входните стойности, изпълнява операцията и ще отпечата резултата в същия блок.

Функция с аргументи и без тип за връщане

Аргументите се предават през извикването на функцията, но изходът не се връща към основната функция.

Функция без аргументи и с тип за връщане

При този тип не се предава аргумент чрез извикването на функцията, но изходът се връща към основната функция.

Функция с аргументи и с тип за връщане

В този тип аргументите се предават през извикването на функцията и изходът се връща към основната функция.

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Параметри:

- Параметрите е стойността(ите), записана (и) в скоби, когато пишем хедъра на функцията.
- Ако се изисква повече от една стойност, те се изброяват в списък, разделени със запетая.

Пример: `def add(a,b):`

В. Аргументи:

Аргументите са стойностите, предоставени в израза за извикване на функция. Списъкът с аргументи трябва да бъде предоставен по същия начин, както са изброени параметрите. Ограничаването на параметрите с аргументите се извършва 1:1, така че трябва да има същия брой и тип аргументи, както е споменато в списъка с параметри.

Пример: `add(a,b)`

Декларация за връщане:

Операторът `return` се използва за излизане от функция и връщане на мястото, откъдето е извикана.

Синтаксис:

```
def add(a,b):  
    c = a + b  
    return(c)  
c=add(2,2)  
print("Result:", c)
```

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Видове аргументи:

Можете да извикате функция, като използвате следните типове формални аргументи:

- *Задължителни аргументи;*
- *Аргументи без ключови думи (*args);*
- *Аргументи с ключови думи (**kwargs)*
- *Аргументи по подразбиране;*

Задължителни аргументи

Броят на аргументите в извикването на функцията трябва да съвпада точно с дефиницията на функцията.

Таблица 3.

<pre>def student(name, roll): print(name,roll) student("george",98) student(101,"Lili")</pre>	<pre>george 98 101 Lili === Code E</pre>
---	---

<pre>def student(name, roll): print(name,roll) student(101)</pre>	ERROR! Traceback (most recent call last): File "<main.py>", line 3, in <module> TypeError: student() missing 1 required positional argument: 'roll'
---	---

<pre>def student(name, roll): print(name,roll) student(101)</pre>	student() missing 1 required positional argument: 'name' <i>//student() липсва 1 задължителен позиционен аргумент: 'name'</i>
<pre>def student(name, roll): print(name,roll) student()</pre>	student() missing 2 required positional arguments: 'roll' and 'name' <i>//student() липсват 2 задължителни позиционни аргумента: 'roll' и 'name'</i>

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Интерпретаторът на Python може да използва предоставените ключови думи, за да съпостави стойностите с параметрите, дори ако те са подредени неправилно

Таблица 4.

```
def student(name, roll, mark):  
    print(name, roll, mark)  
student (mark=90, roll=102, name="bala")
```

bala 102 90

=== Code Execution Successful ===

Аргументи без ключови думи (*args)

В Python ***args** се използва за предаване на променлив брой аргументи към функция.

Използва се за предаване на списък с аргументи с променлива дължина, без ключови думи. Тези аргументи се събират в кортеж във функцията и ни позволяват да работим с тях.

Пример: Да се дефинира функция `sum_all`, която приема произволен брой аргументи. Синтаксисът `*args` събира всички аргументи в кортеж с име `args`. Вътре във функцията се преминава през кортеж `args` и се изчислява сумата от всички числа.

```
def sum_all(*args):  
    result = 0  
    for num in args:  
        result += num  
    return result  
  
print(sum_all(1, 2, 3, 4, 5))
```

Резултат: 15

Аргументи с ключови думи (**kwargs)

В Python ****kwargs** се използва за предаване на списък с аргументи с ключови думи и променлива дължина. Двойната звезда позволява да се предават аргументи на ключови думи (в произволен ред). Аргументите се събират в речник във функцията, което позволява достъп до тях чрез техните ключове.

```
def display_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
display_info(name="Alice", age=30, city="New York")
```

```
name: Alice  
age: 30  
city: New York
```

```
=== Code Execution Successful ===
```

Аргументи по подразбиране

Приема стойност по подразбиране, ако стойността не е предоставена в извикването на функцията за този аргумент.

Таблица 5.

```
def student(name="ala", roll=101, mark=50):  
    print(name,roll,mark)  
student ()  
student (name="bala")  
student (name="bala",roll=102)  
student (mark=90,roll=102,name="bala")
```

```
ala 101 50  
bala 101 50  
bala 102 50  
bala 102 90
```

```
=== Code Execution Successful ===
```

2. Операции върху низ

Низът е тип данни, използван в програмирането за представяне на текст. Състои се от набор от знаци, като масив от символи се използва за съхраняване на низ.

Пример: „Lili“, „2345“

Основните операции върху низа са:

- ☐ Индексиране (**Indexing**)
- ☐ Нарязване (**Slicing**)
- ☐ Конкатенация (**Concatenation**)
- ☐ Повторения (**Repetitions**)
- ☐ Членство (**Membership**)

също така:

- ☐ дължина на низ (преброяване на броя на символите в низа),
- ☐ обръщане на низ,
- ☐ сравняване на два низа.

В Python аритметичният оператор ‘+’ се използва за конкатенация на низове, а релационните оператори ‘<’, ‘<=’, ‘>’, ‘>=’ ‘==’ се използват за сравнение на низове.

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Таблица 6.

Indexing Индексиране	>>>a='HELLO' >>>print(a[0]) >>>H >>>print(a[-1]) >>>O	❖ Положителното индексиране помага за достъп до низа в началото. ❖ Отрицателният индекс помага за достъп до низа от края.
Slicing Нарязване	a = 'Hell' print(a[slice(2)]) >>> He a = 'Hell' print(a[slice(1, 2)]) >>> e	Операторът slice[n : m] извлича подниз от низовете. Сегмент от низ се нарича срез. https://www.w3schools.com/python/ref_func_slice.asp
Concatenation Конкатенация	a="save " b="eart h" print(a + b) save eart h	Операторът + свързва текста от двете страни на оператора.
Repetitions Повторения	a="panimalar " print(3*a) panimalar panimalar panimalar	Операторът * повтаря низа от лявата страна, умножен по стойността от дясната страна.
Membership Членство	>>> s="good morning" >>>"m" in s True >>>"a" not in s True	Използване на оператори за членство, за да проверите дали конкретен знак е в низ или не. Връща true, ако той присъства.

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Python не поддържа тип данни за символи. Низ с размер 1 може да се третира като знак.

- ☐ единични кавички (")
- ☐ двойни кавички (" ")
- ☐ тройни кавички ("""" """)

3. Достъп до символи в Python

В Python отделните символи на низ могат да бъдат достъпни с помощта на метода на индексирание. Индексирането позволява отрицателни адресни препратки за достъп до знаци от крайната част на низа, напр. -1 се отнася до последния знак, -2 се отнася до предпоследния знак и т. н. Докато достъпът до индекс извън диапазона води до IndexError.

Само цели числа могат да се предават като индекс, float или друг тип води до TypeError.

Таблица 2.

String A	H	E	L	L	O
Positive Index	0	1	2	3	4
Negative Index	-5	-4	-3	-2	-1

```
main.py
1 a='HELLO'
2 print(a[0])
3 print(a[-1])
4
5 a = 'Hell'
6 print(a[slice(2)])
7 a = 'Hell'
8 print(a[slice(1, 2)])
9
10 a="save "
11 b="eart h"
12 print(a + b)
13
14 a="panimalar "
15 print(3*a)
16
17 s="good morning"
18 print("m" in s)
19 print("a" not in s)
```

Shell

H
O
He
e
save eart:h
panimalar: panimalar panimalar
True
True
>

А. Генериране на случайни ASCII символи в PYTHON

Генериране на произволен низ с фиксирана дължина.

Генериране на произволен низ с малки и главни букви.

Генериране на произволен буквено-цифров низ с букви и цифри.

Генериране на парола с произволен низ, който съдържа буквите, цифрите и специалните знаци.

Извикване на `string.ascii_letters`, за получаване на низ от азбука с малки букви, последвана от азбука с големи букви.

Използване на `random.choice(seq)` с низа от азбуки като `seq`, за да се генерира произволна буква от низа.

Пример:

Генериране на произволна буква

```
import random
import string
```

```
# Генерирайте произволен низ от конкретни букви
lower_upper_alphabet = string.ascii_letters
random_letter = random.choice(lower_upper_alphabet)
print(random_letter)
```

Резултат:

O
k

String методи : str(), upper(), lower(), count(), find(), replace() и len()

len()

Пример:

```
>>> s = "This string contains forty-two characters."  
>>> len(s) 42
```

Пример:

.upper() & .lower() Изпълнението на метода .upper() върху низ преобразува всички знаци в главни букви, докато методът lower() преобразува всички знаци в малки букви

```
>>> s = "Whereof one cannot speak, thereof one must be silent."  
>>> s  
'Whereof one cannot speak, thereof one must be silent.'  
  
>>> s.upper()  
'WHEREOF ONE CANNOT SPEAK, THEREOF ONE MUST BE SILENT.'  
  
>>> s.lower()  
'whereof one cannot speak, thereof one must be silent.'
```


Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Пример:

.count() Методът сумира броя пъти, в които символ или последователност от символи се появяват в низ

```
>>> s = "That that is is that that is not is not is that it it is"
```

```
>>> s.count("t")
```

```
13
```

```
>>> s = s.lower() # преобразуваме низа до низ съставен от малки букви
```

```
>>> s.count("t") # извеждаме броя срещания на символа t
```

```
14
```

```
s = "James while John had had had had had had had had had had a better effect on the teacher"
```

```
>>> s.count("had")
```

```
11
```

.find() Търсим конкретен символ или знаци в низ с метода

Пример:

```
s = "On the other hand, you have different fingers."
```

```
print(s.find("O") )
```

```
0
```

```
s = "On the other hand, you have different fingers."
```

```
print(s.find("hand") )
```

```
13
```

#ПОЗИЦИЯТА ОТ КОЯТО ЗАПОЧВА

```
>>> s.find("o")
```

```
7
```

Но ако искаме да намерим второто „о“, трябва да посочим диапазон.

```
>>> s.find("o", 8)
```

20

Започва търсене от 8-ия елемент и намира „о“ на 20. Можете също така да посочите край на диапазона и, подобно на нарязването, можем да го направим назад:

```
>>> s.find("e", 20, -5)
```

26

.replace() – заменяне на една стойност с друга

```
>>> s = "I intend to live forever, or die trying."
```

```
>>> s.replace("to", "three") 'I intend three live forever, or die trying.'
```

```
>>> s.replace("fore", "five") 'I intend to live fivever, or die trying.,'
```

String Conversion – преобразуване на число в стринг

```
>>> one = str(1)
```

```
>>> one
```

```
'1'
```

Б. Генериране на произволен низ с фиксирана дължина

За да генерираме произволен низ, трябва да използваме следните два модула на Python.

Модулът `string` съдържа различни низови константи, които съдържат ASCII знаците. Модулът `random` съдържа отделни константи за малки, главни букви, цифри и специални знаци.

random модул

Генериране на произволен низ с фиксирана дължина `n`.

Използване на низовата константа `string.ascii_lowercase`, за да получим всички малки букви в един низ. Константата `string.ascii_lowercase` съдържа всички малки букви.

т.е. „abcdefghijklmnopqrstuvwxyz“

Изпълнете цикъл за `n` брой пъти, за да изберете единичен символ от низова константа с помощта на функция `random.choice()` и да го добавите към низовата променлива с помощта на функция за присъединяване.

Забележка: – Функцията `random.choice()`, се използва за избор на единичен знак от списък.

Например, да предположим, че искате произволен низ с дължина 6, след което можем да изпълним функция `random.choice()` 6 пъти, за да изберете една буква от `string.ascii_lowercase` и да я добавите към низовата променлива.

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

В. Генериране на произволен низ само от конкретни букви

```
import random
import string
# Генерирайте произволен низ от конкретни букви
def randString(length=5):
    #поставете в your_letters следното
    your_letters='abcdefghi'
    return ''.join((random.choice(your_letters)
for i in range(length)))
print ("Random String with specific letters ", randString() )
print ("Random String with specific letters ", randString(1) )
print ("Random String with specific letters ", randString(2) )
print ("Random String with specific letters ", randString(3) )
print ("Random String with specific letters ", randString(4) )
print ("Random String with specific letters ", randString(5) )
print ("Random String with specific letters ", randString(10) )
```

Резултат:

Random String with specific letters bihg
Random String with specific letters e
Random String with specific letters db
Random String with specific letters dae
Random String with specific letters hddi
Random String with specific letters ffdfi
Random String with specific letters abhegabagg

```
import random
import string

# Генерирайте произволен низ от конкретни букви
def randString(length=5):
    #поставете в your_letters следното
    your_letters='abcdefghi'
    return ''.join((random.choice(your_letters)
for i in range(length)))

print ("Random String with specific letters ", randString() )
print ("Random String with specific letters ", randString(1) )
print ("Random String with specific letters ", randString(2) )
print ("Random String with specific letters ", randString(3) )
print ("Random String with specific letters ", randString(4) )
print ("Random String with specific letters ", randString(5) )
print ("Random String with specific letters ", randString(10) )
```

Random String with specific letters bihg
Random String with specific letters e
Random String with specific letters db
Random String with specific letters dae
Random String with specific letters hddi
Random String with specific letters ffdfi
Random String with specific letters abhegabagg

=== Code Execution Successful ===

Г. Генериране на произволен низ с букви и цифри в Python

Често се налага да се състави произволен низ, който да съдържа както букви, така и цифри.

Например: ab23cd, jkml98, 87thki.

В случая трябва да използват константите `string.ascii_letters` и `string.digits`, така се получават комбинациите от букви и цифри в произволен низ.

Пример:

```
import random
import string
```

```
def randomStringDigits(stringLength=6):
    """Generate a random string of letters and digits """

    lettersAndDigits = string.ascii_letters + string.digits
    return "".join(random.choice(lettersAndDigits)
for i in range(stringLength))

print ("Generating a Random String including letters and digits")
print ("First Random String is ", randomStringDigits(8))
print ("Second Random String is ", randomStringDigits(8))
print ("Third Random String is ", randomStringDigits(8))
```

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
import random
import string

def randomStringDigits(stringLength=6):
    """Generate a random string of letters and digits """

    lettersAndDigits = string.ascii_letters + string.digits
    return ''.join(random.choice(lettersAndDigits)
for i in range(stringLength))

print ("Generating a Random String including letters and digits")
print ("First Random String is ", randomStringDigits(8))
print ("Second Random String is ", randomStringDigits(8))
print ("Third Random String is ", randomStringDigits(8))
```

```
Generating a Random String including letters and digits
First Random String is  iQQBOM06
Second Random String is  Eimbxgiq
Third Random String is  MgTAjD8T

=== Code Execution Successful ===
```

4. Форматиране на числа и низ

4.1. Базово форматиране

Най-често се използва позиционно форматиране. Прилага се, когато редът на аргументите няма да се променя или има малко елементи, които искате да свържете. Тъй като елементите не са представени с описателно име, този стил **трябва да се използва само за форматиране на относително малък брой елементи.**

Пример:

Old

```
print ( '%s %s' % ('one', 'two'))
```

New

```
print ( '{} {}'.format('one', 'two'))
```

Output: one two

Old

```
print ('%d %d' % (1, 2))
```

New

```
print ('{} {}'.format(1, 2))
```

Output: 1 2

main.py	  	Shell
1	print ('%s %s' % ('one', 'two'))	one two
2	print ('{} {}'.format('one', 'two'))	one two
3	print ('%d %d' % (1, 2))	1 2
4	print ('{} {}'.format(1, 2))	1 2

С новото стилово форматиране е възможно (а в Python 2.6 дори задължително) да се даде на заместителите изричен позиционен индекс. Това позволява пренареждане на реда на показване без промяна на аргументите. *Тази операция не е налична при форматиране в стар стил.*

New

```
print ( '{1} {0}'.format('one', 'two'))
```

Output: two one

4.2. Допълване и подравняване на низове

По подразбиране стойностите са форматиран така, че да заемат само толкова символи, колкото са необходими за представяне на съдържанието. Възможно е дефиниране на стойността, така, че да бъде допълнена тя, до определена дължина.

За съжаление подравняването по подразбиране се различава между стария и нов стил. Старият стил по подразбиране е подравнен вдясно, докато за новия стил е вляво.

А. Подравняване вдясно:

Old

```
print ( '%10s' % ('test',))
```

New

```
print ( '{:>10}'.format('test'))
```

Output:

1	<code>print ('%10s' % ('test',))</code>	test
2	<code>print ('{:>10}'.format('test'))</code>	test

Б. Подравняване вляво:

Old

```
print ( '%-10s' % ('test',))
```

New

```
print ( '{:10}'.format('test'))
```

Output: test

Отново новото стилово форматиране превъзхожда стария вариант, като предоставя повече контрол върху това как стойностите са подравнени. Можете да изберете символа за допълване. *Тази операция не е налична при форматиране в стар стил.*

New

```
print ( '{:_<10}'.format('test'))
```

test

	test
1	<code>print ('{:>10}'.format('test'))</code> test
2	<code>print ('{:10}'.format('test'))</code> test
3	<code>print ('{:_<10}'.format('test'))</code> test_____

В. Централно подравняване:

Тази операция не е налична за форматиране в стар стил.

New

```
print('{:^10}'.format('test'))
```

Output: test

Когато използвате централно подравняване, където дължината на низа води до неравномерно разделяне на знаците за допълване, **допълнителният знак ще бъде поставен от дясната страна.** *Тази операция не е налична при форматиране в стар стил.*

New

```
print('{:^6}'.format('zip'))
```

Output: zip

Г. Съкращаване на дълги низове (Truncating long strings)

Обратно на допълването също е възможно да се съкратят твърде дълги стойности до определен брой символи. Числото **зад точката (.)** във формата определя прецизността на изхода. За низове това означава, че изходът е съкратен до определената дължина. В нашия пример това ще бъде 5 знака.

Old

```
print('%0.5s' % ('xylophone',))
```

New

```
print('{:.5}'.format('xylophone'))
```

Output: xylop

Д. Комбиниране на съкращаване и запълване (Combining truncating and padding)

Възможно е също да комбинирате съкращаване и запълване:

Old

```
print ( '%-10.5s' % ('xylophone',))
```

New

```
print ( '{:10.5}'.format('xylophone'))
```

Output: xylop

4.3. Цели числа:

Разбира се, също така е възможно да се форматират числа.

Old

```
print ('%d' % (42,))
```

New

```
print ('{:d}'.format(42))
```

Output: 42

4.4. Числа с плаваща запетая:

Old

```
print ('%f' % (3.141592653589793,))
```

New

```
print ('{:f}'.format(3.141592653589793))
```

Output: 3.141593

А. Допълващи числа (Padding numbers)

Подобно на низовете, числата също могат да бъдат ограничени до определена ширина.

Old

```
print ('%4d' % (42,))
```

New

```
print ('{:4d}'.format(42))
```

Output: 42

Отново подобно на съкращаването на низове, точността на числата с плаваща запетая ограничава броя на позициите след десетичната запетая. За числа с плаваща запетая, стойността на допълване представлява дължината на пълния изход. В примера по-долу искаме изходът ни да има поне 6 знака, като 2 са след десетичната запетая.

Old

```
print ('%06.2f' % (3.141592653589793,))
```

New

```
print ('{:06.2f}'.format(3.141592653589793))
```

Output: 003.14

За целочислени стойности предоставянето на точност няма много смисъл и всъщност е забранено в новия стил (това ще доведе до ValueError).

Old

```
print ('%04d' % (42,))
```

New

```
print ('{:04d}'.format(42))
```

Output: 0042

```
print ('{:4d}'.format(42))
```

42

```
print ('{:04d}'.format(42))
```

0042

Б. Подписани числа (Signed numbers)

По подразбиране само пред отрицателни числа се поставя знак. Това разбира се може да се промени.

```
Old  
print ('%+d' % (42,))  
New  
print ('{:+d}'.format(42))  
Output: +42
```

Използвайте символ за интервал, за да посочите, че отрицателните числа трябва да имат префикс със символ минус и **ведещ интервал трябва да се използва за положителни.**

```
Old  
print ('% d' % ((- 23),))  
New  
print ('{: d}'.format((- 23)))  
Output: -23
```

```
print ('% d' % (42,))  
Old  
print ('{: d}'.format(42))  
New  
Output: 42
```

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Новото стилово форматиране също така може да контролира позицията на символа на знака спрямо допълването. Тази операция не е налична при форматиране в стар стил.

New

```
print('{:=5d}'.format((- 23)))
```

Output: - 23

New

```
print('{:=+5d}'.format(23))
```

Output: + 23

```
print ( '{:=+5d}'.format(23))
```

+ 23

В. Именувани заместители (Named placeholders)

И двата стила на форматиране поддържат именувани заместители.

Setup

```
data = {'first': 'Hodor', 'last': 'Hodor!'}
```

Old

```
print ( '%(first)s %(last)s' % data)
```

New

```
print ( '{first} {last}'.format(**data))
```

Output: Hodor Hodor!

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

`.format()` също приема аргументи на ключови думи. Тази операция не е налична при форматиране в стар стил.

New

```
print('{first} {last}'.format(first='Hodor', last='Hodor!'))
```

Output: Hodor Hodor!

На следващият слайд е представен пример с низове. Реализирани са няколко функции за:

- 1) Връщане на броя на символите в зададен низ (т.е. дължина на низ)
- 2) Връщане на обърнат на зададеният низ
- 3) Връщане на конкатенацията от два низа
- 4) Сравняване на два низа

- 1) Връщане на броя на символите в зададен низ (т.е. дължина на низ)
- 2) Връщане на обърнат на зададеният низ
- 3) Връщане на конкатенацията от два низа
- 4) Сравняване на два низа

Пример:

```
def strlen(str):
```

```
    counter = 0
```

```
    while str[counter:]:
```

```
        counter += 1
```

```
    return counter
```

```
def strrev(str):
```

```
    rstr=""
```

```
    l=strlen(str)
```

```
    while l>0:
```

```
        rstr = rstr + str[l-1]
```

```
        l=l-1
```

```
    return rstr
```

```
def strcat(st1,st2):
```

```
    return(st1+st2)
```

```
def strcmp(st1,st2):
```

```
    if(st1==st2):
```

```
        print(st1 + " and " + st2 + " are same")
```

```
    elif (st1>st2):
```

```
        print(st1 + " comes after " + st2 + " in the Dictionary")
```

```
    else:
```

```
        print(st1 + " comes before " + st2 + " in the Dictionary")
```

```
print("Functions:\n 1. Length \n 2. Reverse \n 3. Concatenation\n" +  
    " 4. Comparison\n")
```

```
n=int(input("Enter your Choice: "))
```

```
if(n==1):
```

```
    str = input("Enter a String: ")
```

```
    print("Length of the string is:", strlen(str))
```

```
elif (n==2):
```

```
    str = input("Enter a String: ")
```

```
    print("Reversed String is:", strrev(str))
```

```
elif (n==3):
```

```
    str1 = input("Enter the first String: ")
```

```
    str2 = input("Enter the second String: ")
```

```
    print("Concatenated string is:",
```

```
    strcat(str1,str2))
```

```
elif (n==4):
```

```
    str1 = input("Enter the first String: ")
```

```
    str2 = input("Enter the second String: ")
```

```
    strcmp(str1,str2)
```

```
    else:
```

```
        print("Invalid Choice")
```

Тема 3_1. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Functions:

1. Length
2. Reverse
3. Concatenation
4. Comparison

Enter your Choice: 1
Enter a String: xхагда
Length of the string is: 6

Functions:

1. Length
2. Reverse
3. Concatenation
4. Comparison

Enter your Choice: 2
Enter a String: асдф
Reversed String is: фдса

Functions:

1. Length
2. Reverse
3. Concatenation
4. Comparison

Enter your Choice: 3
Enter the first String: дгхас
Enter the second String: даса
Concatenated string is: дгхасдаса

Functions:

1. Length
2. Reverse
3. Concatenation
4. Comparison

Enter your Choice: 4
Enter the first String: хдда
Enter the second String: хдда
хдда and хдда are same

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009